

Proofs at the Scale of Ideas

An Intent-Directed, Lean-Checked Proof Language

A proposal inspired by ProveIt and Leant

Prepared for Vladimir Reshetnikov

September 5, 2026

Abstract

A useful proof language should let an author say what makes an argument work without repeatedly programming the administrative details of its verification. This article proposes OUTLINE, a working name for a controlled mathematical language embedded in Lean. Its basic unit is a *semantic checkpoint*: a precisely stated intermediate fact or a certified proof plan such as induction, symmetry reduction, finite reindexing, or a derivative-bound argument. Omitted details become scoped proof obligations, not implicit axioms. Search may use ordinary tactics, structural term synthesis, theorem retrieval, or an external model; acceptance requires a proof of the already fixed Lean statement and an explicit audit of its assumptions.

The proposal is grounded in selected ProveIt modules and in Leant’s candidate generation, verification, ranking, and advisory tactic infrastructure. Worked studies include residual-to-root certification, Gaussian-binomial summation, commutation by induction, and a floor-square-root sum. They distinguish cosmetic abbreviation from genuine mathematical reorganization. A small core calculus, a relative soundness argument, contracts for higher-level plans, a reproducible artifact format, an implementation roadmap, and a controlled evaluation design make the proposal concrete. The language is not implemented here; syntax examples are specifications, and no compilation or performance results for them are claimed.

Central design decision

Make the *mathematical structure* explicit and the *verification plumbing* implicit—but retain a checkable account of both. A short text is not an acceptable improvement if it proves a different theorem, hides an essential hypothesis, or merely moves unbounded search behind the word “obvious.”

Contents

1	The problem is not simply too many characters	3
2	What the ProveIt examples actually suggest	4
3	Position among existing proof languages and tools	5
4	The language contract	6
5	A surface language organized around checkpoints	7
6	Worked study I: a residual certifies a nearby root	9
7	Certified plans for familiar mathematical moves	11
8	Worked study II: Gaussian coefficients and finite summation	13
9	Worked study III: exact arithmetic through double counting	15
10	A small core calculus and a relative soundness argument	17
11	A synthesis architecture inspired by Leant	19
12	Trust, semantic fidelity, and reproducible artifacts	21
13	The authoring experience: progressive disclosure without concealment	23
14	What should be optimized, and how to test it	24
15	An implementation path that can be attempted incrementally	26
16	Conclusion	28
A	Source inventory and reproducibility notes	29
B	A minimal grammar and obligation record	30
C	Evidence states and a negative-test specification	31
	References	33

1 The problem is not simply too many characters

A mathematical proof and a proof-assistant script are often organized around different questions. The mathematician asks: What quantity should be introduced? What invariant should be preserved? Which symmetry removes a case? Which theorem connects the hypotheses to the desired conclusion? A low-level script must additionally settle coercions, argument order, membership certificates, rewrite direction, induction motives, and the exact shape of the next goal.

Some of these details are genuinely routine. Others only *look* routine because an informal argument has silently changed its domain, strengthened an assumption, or used an invalid cancellation. A language that indiscriminately suppresses details would therefore solve the wrong problem. The target is not maximum terseness, and certainly not “natural language in, theorem out.” The target is a better division of labor between the author and a proof-producing elaborator.

OUTLINE is proposed as a language of *mathematical commitments*. The author commits to the statement, meaningful intermediate facts, and major transformations. The elaborator reconstructs the local evidence required to connect them. A successful reconstruction is saved, checked, and inspectable; a failed reconstruction identifies the unsatisfied commitment rather than silently changing its meaning.

A one-line invocation of an existing theorem is already excellent when the theorem is the appropriate abstraction. Conversely, a one-line call to an enormous search procedure can be a poor explanation. Source length, proof-term size, search effort, and explanatory quality must be evaluated separately.

1.1 What was examined, and what was not

This is a targeted source review, not a complete audit or a statistical survey. The ProveIt sample consists of `Basic.lean`, `MeanValueBracket.lean`, `QPascalSummation.lean`, early arithmetic lemmas in `Arithmetic.lean`, and `FloorSqrtSum.lean`. Together they expose subtype projections, order cases, analytic side conditions, finite-index manipulation, generalized induction, and exact arithmetic transport. The source inventory in [section A](#) identifies the declarations and pinned files.[\[2, 3, 4, 5, 6\]](#)

The Leant review includes its overview, selected synthesis and suggestion code in `src/Main.hs`, the candidate-verification boundary, fragment representation, candidate-quality documentation, and history-replay helper. The inspected revisions are:

Repository	Commit used for source references
ProveIt	24ce8bd743eaab64a91ce90725ea00f498d319d2
Leant	c36adf114035fb2fba6c276b63944cc613b31668

These commit identifiers were resolved from the public repository histories during the review. They identify the inspected snapshots, not a claim that a live branch cannot subsequently change.[\[1, 7\]](#)

Neither repository was rebuilt for this article. No proposed OUTLINE program was run through a compiler, and no purported compression factor is reported. The mathematical arguments below are developed explicitly; their suggested encodings remain design examples. This distinction is important: a plausible language surface does not demonstrate a reliable elaborator.

1.2 Four different sources of improvement

There are at least four independent interventions. *Library engineering* supplies the right theorem in the first place. *Proof refactoring* finds a cleaner argument. *Automation* discharges predictable obligations. *Language and interface design* express and expose these activities at the right level. OUTLINE should coordinate them, but should not take credit for all of them.

For example, replacing a long derivative calculation with a named mean-value lemma is mostly a library improvement. Replacing two sign cases by simultaneous endpoint estimates changes the proof. Inferring that the endpoints belong to a containing interval is automation. Letting the author say “apply the lower slope bound on these two intervals” is a language feature. A fair evaluation must preserve these distinctions.

2 What the Provelt examples actually suggest

2.1 Subtype access should be cheap; mathematical identity should not

In `Basic.lean`, a bounded Fabius function takes values in the subtype $[0, 1]$. The real-valued view exposes that value, while its nonnegativity and upper bound follow from the subtype’s stored property. The module also distinguishes this bounded function from a signed smooth extension. Its derivative assumption has a specified interval of validity. These are useful examples of *safe projection* versus *unsafe identification*.^[2]

An elaborator should make the consequences of $u \in [0, 1]$ readily available, without forcing the author to name both conjunction projections. It must not conclude that two functions with related names are interchangeable globally. A change from a bounded cumulative function to a signed extension is a mathematical transport that needs a theorem and a domain, not an invisible coercion.

The resulting design principle is narrow but powerful: automatically expose certified local information attached to an object; do not automatically replace the object by a merely analogous one.

2.2 Symmetric order cases can be represented once

The upper absolute-value estimate in `MeanValueBracket.lean` applies one-sided derivative bounds after splitting the order of two points, then normalizes absolute values in each branch. The same module’s inverse-error corollaries are already short applications and rewrites. Thus the sample contains both repetitive plumbing and effective existing abstractions.^[3]

The language opportunity is a *checked symmetry reduction*. The author can specify the exchange of two variables and prove the ordered case once. The elaborator must prove coverage and transport of the complete context. This is not the same as trusting the phrase “without loss of generality.”

2.3 Finite sums need an index language, not more rewrite guesses

The Gaussian-binomial summation module uses a recurrence, splits a finite range, shifts an index, and removes a vanishing boundary coefficient. It also proves a commutation property by induction in the row index while generalizing the column index. One summation theorem is stated over a semiring; its alternative form uses a commutative semiring.^[4]

These are recognizable mathematical operations, yet their formal implementation depends on the particular range presentation. A better surface should name the range transformation and preserve its boundary obligations. It should also keep the distinction between additive rearrangement and reordering noncommuting products.

2.4 Arithmetic transport has real preconditions

The floor-square-root module proves a rational version of a summation formula and transfers it to natural numbers. It explicitly establishes divisibility by 6 and a bound that makes natural subtraction agree with ordinary subtraction. The proof separates unchanged and jumping values of the integer square root.[6]

The presence of cast lemmas is not evidence that the conditions can be erased. The conditions are precisely what distinguish a valid transport from a false one. The goal should be to infer or derive them once, show them on demand, and refuse an invalid transport.

The early arithmetic part of `Arithmetic.lean` similarly builds identities from recurrences for binomial coefficients and from the separation of even and odd factors in a factorial. Such examples invite reusable recurrence and finite-product plans. They do not require a new foundational logic.[5]

2.5 A taxonomy of proof effort

Kind of effort	Usually safe to reconstruct	Must remain accountable
Representation	Subtype projections, definitional unfolding, routine casts	The chosen representation and every non-definitional bridge
Local calculation	Ring normalization, interval arithmetic, propositional assembly	Algebraic structure, denominator conditions, order direction
Index management	Bijection arithmetic, finite boundary evaluation	Index map, support, multiplicity, exceptional terms
Structural reasoning	Branch plumbing, recursor applications	Induction motive, generalized variables, symmetry action
Mathematical discovery	Search suggestions for useful lemmas	The accepted intermediate claim or proof plan

Table 1: The proposed boundary is semantic, not a threshold on the number of Lean lines.

3 Position among existing proof languages and tools

Readable formal proofs and proof-producing automation are established ideas. A credible proposal must build on them rather than rediscover them under a new name.

Isar already treats a proof as a structured document with explicit contexts and intermediate conclusions, while permitting tactic-style methods inside that structure. Mizar provides a mathematical language and a checker for texts written in it. They are precedents for separating the proof’s readable organization from low-level inference management.[15, 16]

Naproche-SAD’s ForTheL uses a restricted language designed to resemble mathematical text. Verbose Lean is particularly close to the present setting: it supplies controlled-natural-language commands and tactics within Lean, with a pedagogical emphasis on producing proofs that transfer to paper. These examples support using a controlled language, but they do not imply that unconstrained prose can be given a unique intended semantics.[17, 18, 19]

Aesop supplies extensible proof search inside Lean. Sledgehammer combines relevance-guided external search with reconstruction in Isabelle; its manual also discusses generated structured proofs and reconstruction failure. Bundy’s proof-planning program treats recurring proof patterns as plans that guide lower-level inference. These are direct ancestors of several mechanisms proposed here.[20, 21, 22]

The 2026 Apply2Isar preprint studies conversion from apply-style proofs to structured Isar. Its relevance here is architectural: exploration history and readable final presentation need not have

the same form. This article does not claim the invention of that separation.[23]

The intended contribution of OUTLINE is instead an integrated design with three priorities: *statement locking before search*, *certified mathematical plans with visible obligations*, and *search-free replay of accepted evidence*. Leant provides a useful experimental foundation for the middle of this architecture, especially the distinction between candidate generation and acceptance. The added ambition is a stable source language whose mathematical commitments survive both automation and later maintenance.

Precedent	Lesson to retain	Proposed emphasis in OUTLINE
Isar / Mizar	Structured readable formal text	Lean-native statements and explicit semantic-plan contracts
ForTheL / Verbose Lean	Controlled mathematical phrasing	Concision through reconstruction, not merely English synonyms
Aesop / Sledgehammer	Search separated from proof validation	Per-checkpoint budgets, evidence receipts, frozen replay
Proof planning	Reusable high-level reasoning methods	User-authored plans with a checked reconstruction interface
Leant	Candidate synthesis, ranking, verified suggestions	A typed obligation graph and intent-aware final artifacts

Table 2: A comparison of design emphasis, not a claim that any listed system lacks all the other features. Sources are discussed in the surrounding text.

4 The language contract

4.1 Three representations of one proof

The first representation is the *outline*: the author’s statement, local notation, essential intermediate claims, and major proof plans. The second is the *obligation graph*: every local goal together with its exact context, proposed justification, dependencies, and source span. The third is the *checked artifact*: concrete Lean declarations and proofs, tied to a fixed environment and accompanied by an audit record.

$$\boxed{\text{mathematical outline}} \longrightarrow \boxed{\text{scoped obligations}} \longrightarrow \boxed{\text{Lean proof artifact}}.$$

The arrows are not steps that increase trust. The outline-to-obligation translator and all search procedures may be buggy. The final acceptance path must validate the reconstructed term against the original, already elaborated theorem statement.

The graph is useful for explanation and incremental checking. The term is the logical evidence. A hash, a green interface badge, a synthesis-engine receipt, or a graph edge is not a substitute for that evidence.

4.2 Lock the statement before attempting its proof

The theorem header is elaborated in a restricted, declared environment before proof search starts. All types, instances, notation, coercions, and implicit arguments that determine its meaning are resolved. The resulting Lean expression is recorded as the theorem’s target. The proof phase cannot choose a different interpretation because it happens to be easier to prove.

For example, the parser must not decide whether $n - 1$ is natural, integer, or real subtraction by trying all three interpretations and retaining the provable one. It may use the declared type of n , an expected type, or an explicit annotation. If genuinely distinct interpretations remain, it

should ask the author to choose. Definitionally equal presentations need not trigger a question; mathematically different statements do.

A readable statement view should show the selected number systems and overloaded operations. A more detailed view should reveal the fully elaborated type and referenced definitions. Statement locking prevents a class of proof-driven reinterpretations, but it cannot prove that the user intended what the statement says. Human review of definitions remains necessary.

4.3 Omissions are questions, not assumptions

A missing proof of $a \neq 0$ may be solved from existing hypotheses, left as a visible obligation, or reported as a missing premise. It must not become a new hypothesis of the theorem. Likewise, an unfinished claim cannot be accepted by silently generalizing it into the theorem’s parameter list.

A declaration marked as a conjecture may retain holes, but it is not published as a checked theorem. “Checked with assumptions” is a different status from “checked under the declared standard axiom policy,” and the additional assumptions must be displayed.

4.4 Plans are stronger than decorative hints

The language distinguishes three forms of guidance:

```
have C by lower_slope on [a,b]      -- a required plan
have C prefer lower_slope          -- a search preference
have C using only h1, h2           -- a restricted fact scope
```

A required plan obliges the elaborator to construct the claimed transformation and its child goals. A preference affects ranking but does not certify how the proof was obtained. A restricted fact scope limits eligible dependencies, subject to an explicitly declared background library and foundational policy.

This distinction matters for explanation. A prover that ignores “by induction” and solves the goal by an unrelated theorem may be logically correct, but has not checked the author’s proposed induction argument. OUTLINE should report the difference, not present both outcomes under the same badge.

5 A surface language organized around checkpoints

5.1 Keep formulas formal; make the connective tissue mathematical

The initial implementation should reuse Lean terms and types rather than invent an unrelated expression language. A small controlled grammar should surround them with familiar proof constructions. The following blocks use an ASCII presentation of proposed syntax; they are specifications, not executable Lean.

```

theorem example_name (parameters) : target
proof
  let auxiliary := expression
  have key_fact : formula by plan using named_facts
  obtain witness with property from existence_fact
  suffice simpler_goal via reduction
  conclude target by local_calculation
end

```

The displayed notation can be mathematical, but its abstract syntax must be exact. A chained assertion $a \leq b \leq c$ expands to a conjunction with a single shared middle expression. An interval such as `Icc(a,b)` denotes a closed interval in the already known ordered type. A quantifier binds a variable with a definite scope. The phrase `by local_calculation` names a versioned, bounded reconstruction profile, not an unlimited promise to solve an arbitrary theorem.

There should be an escape hatch for native Lean proof blocks. It allows gradual adoption and avoids forcing a mature library into a prematurely rigid grammar. The same target locking and final audit apply across the boundary.

5.2 Scoped mathematical context

A context block may package repeated hypotheses such as convexity, continuity, and derivative bounds. Its expansion remains visible in the theorem statement. Named structures are preferable to a global bag of facts that search can interpret unpredictably.

```

context lower_slope_setting
  D : Set Real
  f : Real -> Real
  m : Real
  assume convex(D)
  assume continuous_on(f,D)
  assume differentiable_on(f,interior(D))
  assume forall t in interior(D): m <= derivative(f,t)
end

```

This is a context template, not a new axiom. Instantiating it either introduces exactly these hypotheses into an explicit theorem header or supplies proofs of them from the surrounding context. It does not silently assert their existence for an arbitrary function.

A context index should record membership, order bounds, continuity restrictions, equalities, and available lemma instances using Lean expressions and local-variable identities. It may expose useful consequences lazily. It should not eagerly saturate every possible consequence: unbounded forward reasoning can create enormous irrelevant contexts and make proofs unstable.

5.3 Anaphora without ambiguity

Words such as “this,” “hence,” and “the preceding inequality” can improve readability only when their referents are deterministic. The first implementation should give them lexical rules: `this` refers to the last completed named or anonymous assertion in the current block; branching resets the reference; a compound assertion is not silently projected unless the expected type identifies the projection uniquely.

The editor should show the resolved fact on hover. A changed reference after an edit should appear in the semantic diff. Search should never choose among several plausible referents merely

because one closes the goal.

5.4 From suggestion to accepted source

An author may write a checkpoint and request candidate continuations. Suggestions should be tested on an isolated copy of the local proof state. Accepting one inserts a source-level plan or a concrete proof fragment; merely displaying it must not mutate the proof. This follows the useful separation already present in Leant’s advisory suggestion mechanism.[9]

The essential difference from an autocomplete system is that the accepted continuation has a type, a target, a scope, and a recorded justification. The author remains responsible for selecting a mathematically useful continuation rather than merely the shortest one.

6 Worked study I: a residual certifies a nearby root

6.1 The mathematical task

The declaration `exists_eq_in_residual_interval` is a particularly informative case: it proves existence of a nearby solution, rather than assuming an inverse value already exists. Its source proof introduces the residual radius and then treats the two signs of the residual separately.[3]

Here is the mathematical statement, with all essential hypotheses retained.

Theorem 6.1 (Residual-radius existence). *Let $D \subseteq \mathbb{R}$ be convex. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be continuous on D and differentiable on $\text{int } D$. Suppose $m > 0$ and*

$$m \leq f'(t) \quad (t \in \text{int } D).$$

For $x, z \in \mathbb{R}$, put $\rho = |f(x) - z|/m$. If $[x - \rho, x + \rho] \subseteq D$, then there is $y \in [x - \rho, x + \rho]$ with $f(y) = z$.

The derivative is not required at boundary points of D . The interval containment implies that x and both interval endpoints belong to D ; these need not be added as independent hypotheses.

6.2 A proof organized around both endpoints

We first isolate the mathematical bridge used by the plan.

Lemma 6.2 (Lower slope bound). *Under the convexity, continuity, differentiability, and lower derivative assumptions above, if $u, v \in D$ and $u \leq v$, then*

$$m(v - u) \leq f(v) - f(u). \tag{6.1}$$

The conclusion does not require $m > 0$.

Proof. If $u = v$, both differences are zero. If $u < v$, convexity gives $[u, v] \subseteq D$. Every point of (u, v) lies in the interior of D : a sufficiently small open interval around it is contained in (u, v) . Thus f is continuous on $[u, v]$ and differentiable on (u, v) . The mean value theorem gives $c \in (u, v)$ such that $f(v) - f(u) = f'(c)(v - u)$. Multiplying $m \leq f'(c)$ by $v - u > 0$ proves the result. $\square$

Proof of theorem 6.1. Set $a = x - \rho$ and $b = x + \rho$. Since $m > 0$, we have $\rho \geq 0$, $a \leq x \leq b$, and

$$m\rho = |f(x) - z|. \tag{6.2}$$

The containment assumption puts a, x, b in D . Apply Lemma 6.2 on $[a, x]$ and $[x, b]$. It follows that

$$f(a) \leq f(x) - m\rho = f(x) - |f(x) - z| \leq z, \quad (6.3)$$

$$z \leq f(x) + |f(x) - z| = f(x) + m\rho \leq f(b). \quad (6.4)$$

The middle inequalities use the two elementary bounds $-|r| \leq r \leq |r|$ with $r = f(x) - z$. Continuity on D restricts to $[a, b]$. The intermediate value theorem now supplies $y \in [a, b]$ with $f(y) = z$. This also covers $\rho = 0$: then $f(x) = z$ and the interval is the singleton $\{x\}$. $\square$

This reorganization does not merely hide the original sign split: it avoids needing that split. A prototype could implement the proof directly in Lean without any new language. OUTLINE's separate contribution is expressing the argument in terms of the two slope applications and their common endpoint bracket.

6.3 Proposed source

With the theorem header fixed as in theorem 6.1, the following is a plausible body:

```
proof
  let rho := abs(f(x)-z)/m
  let a := x-rho
  let b := x+rho
  have geometry : a <= x <= b and Icc(a,b) subset D
    by interval_arithmetic
  have scale : m*rho = abs(f(x)-z) by field_arithmetic
  have slopes : f(a) <= f(x)-m*rho
                and f(x)+m*rho <= f(b)
    by lower_slope on [a,x] and [x,b]
  have bracket : f(a) <= z <= f(b)
    by absolute_value_bounds using slopes, scale
  obtain y in Icc(a,b) with f(y)=z
    by intermediate_value using bracket
  finish with y
end
```

Listing 1: Proposed OUTLINE syntax for the endpoint proof.

`finish with y` constructs the existential witness together with the membership and equality evidence introduced by `obtain`. It is not an instruction to use the bare real number as a proof. The terms `interval_arithmetic`, `field_arithmetic`, `lower_slope`, and `intermediate_value` denote distinct provider contracts. In particular, linear arithmetic alone is not being asked to invent an analytic theorem or a fact about absolute values.

6.4 The obligations that are hidden from the default view

The lower-slope provider can target the existing Mathlib declaration used by the source module, `Convex.mul_sub_le_image_sub_of_le_deriv`. The intermediate-value provider can target `intermediate_value_Icc`. These are concrete lowering candidates, not a claim that the proposed surface has already been compiled against the pinned environment.[3]

6.5 Why the side conditions cannot be guessed away

Dropping interval containment is unsound even when the function has a positive derivative wherever the assumptions require it. For example, take $D = [0, 1]$, $f(t) = \min\{1, \max\{0, t\}\}$,

Checkpoint	Required evidence	Intended source
Geometry	$\rho \geq 0$, $a \leq x \leq b$, containment	$m > 0$, absolute-value nonnegativity, hypothesis substitution
Scale	$m \neq 0$, cancellation in $\mathbb{R}$	Positivity and field identities
Slope at left	$a, x \in D$, $a \leq x$, analytic assumptions	Geometry, containment, lower-slope theorem
Slope at right	$x, b \in D$, $x \leq b$, analytic assumptions	The same fixed analytic context
Bracket	$f(x) - f(x) - z \leq z \leq f(x) + f(x) - z $	Absolute-value bounds and ordered arithmetic
Existence	$a \leq b$, continuity on $[a, b]$, endpoint bracket	Restriction of continuity and the intermediate value theorem

Table 3: The proposed default display suppresses this bookkeeping, but every row must acquire a proof.

$m = 1$, $x = 0$, and $z = 2$. The function equals t on D and has derivative 1 on its interior, but it never attains 2 anywhere. The residual interval is not contained in D .

Dropping positivity of m breaks the interpretation of the residual radius and the cancellation step. A separate upper absolute-value theorem also cannot infer $|f(x) - f(y)| \leq M|x - y|$ merely from $m \leq f' \leq M$: $f(t) = -t$, $m = -1$, and $M = 0$ already contradict that conclusion for distinct points. These are mathematical counterexamples to overaggressive elaboration, not failures that a more determined search procedure should bypass.

The diagnostic should therefore identify a specific missing obligation—for example, “the proposed interval is not known to lie in D ”—rather than increasing the search budget or inventing an inverse.

7 Certified plans for familiar mathematical moves

7.1 A plan is a theorem-shaped interface

A proof plan receives an elaborated goal, a local context, and explicit parameters such as an index map or an induction variable. It returns a finite collection of typed subproblems and a reconstruction expression. The expression explains how solutions of those subproblems prove the original goal. It is checked after the subproblems are solved.

For a simple, nondependent plan this looks like a term of type

$$Q_1 \rightarrow Q_2 \rightarrow \cdots \rightarrow Q_r \rightarrow C.$$

For a plan introducing witnesses or branch-local variables, this notation is inadequate. Its inputs are dependent functions over the appropriate local contexts. For instance, existential elimination needs a continuation $\forall x, P(x) \rightarrow C$, not independent global obligations $P(x)$ and C with a free, unexplained x .

A plan library is consequently closer to a library of theorem schemas with elaboration code than to a dictionary of persuasive phrases. The elaboration code may choose a useful instantiation; it may not bypass the schema’s premises.

7.2 Without loss of generality: transform the context as well

Here is an abstract contract for a two-case symmetry reduction.

Proposition 7.1 (A checked reduction to a representative case). *Let U be a type of problem*

states, let $H, R, C : U \rightarrow \text{Prop}$, and let $\sigma : U \rightarrow U$. Suppose that, for every u with $H(u)$:

$$R(u) \vee R(\sigma u), \quad H(\sigma u), \quad C(\sigma u) \rightarrow C(u).$$

Then a proof of $\forall u, H(u) \rightarrow R(u) \rightarrow C(u)$ yields a proof of $\forall u, H(u) \rightarrow C(u)$.

Proof. Fix u and a proof of $H(u)$. If $R(u)$, apply the representative-case proof directly. Otherwise coverage supplies $R(\sigma u)$. Context preservation supplies $H(\sigma u)$, so the representative-case proof gives $C(\sigma u)$. The transport implication gives $C(u)$. $\square$

An involution is a common source of these premises, but it is not required by the abstract proposition. More general normal-form reductions and finite symmetry groups can use analogous coverage and transport certificates.

For the derivative estimate, take $u = (x, y)$, $\sigma(x, y) = (y, x)$, and $R(x, y)$ to mean $x \leq y$. Membership of both points in the same domain is preserved. The conclusion is unchanged under the exchange because absolute differences are symmetric. The resulting source might be:

```
proof
  by symmetry swap(x,y), assume x <= y
  have m*(y-x) <= f(y)-f(x) <= M*(y-x)
    by derivative_bounds on [x,y]
  conclude abs(f(x)-f(y)) <= M*abs(x-y)
    by absolute_value_bounds
end
```

The nonnegativity hypothesis on m is available here because this is the *upper* absolute-value estimate. A lower estimate should not acquire an unnecessary sign assumption merely because it shares a plan implementation.

An asymmetric hypothesis such as $x = 0$ is not generally preserved by swapping x and y . In that situation the plan must fail unless an appropriate context transport is proved. A target's visual symmetry is insufficient. The diagnostic should name the untransported hypothesis.

7.3 Induction: infer plumbing, not the motive by fiat

An induction command should expose its motive and its generalized variables as semantic data. For a statement $P(n, k)$, induction in n while holding k fixed yields a hypothesis $P(n, k)$; it does not yield $P(n, k + 1)$. The stronger motive $\forall k, P(n, k)$ is a mathematical choice.

The editor may suggest that choice when recursive use requires it. Acceptance should update the outline to say **induct n generalizing k**. The generated proof then uses the appropriate recursor with that motive. Nested induction, well-founded induction, and induction over an indexed family need the same discipline, including visible termination or well-foundedness obligations.

A command such as **the step follows from the recurrence** can be useful after the recurrence, motive, and closure rules are fixed. It should not initiate an unbounded search over unrelated induction principles and silently replace the authorial plan with whichever one succeeds.

7.4 Finite reindexing: preserve multiplicity and boundaries

For finite sets A and B and a map $\phi : A \rightarrow B$, a reindexing plan needs evidence that the map is a bijection and that the summands correspond. In a finset-based implementation, this includes

membership, injectivity on the relevant domain, coverage of the target, and the equality of the transported summands.

A noninjective map does not justify reindexing an ordinary sum without accounting for multiplicity. A map that misses one endpoint does not justify dropping that term. A shift $k = j + 1$ therefore exposes the exceptional $k = 0$ term and, when ranges are aligned, any new upper boundary.

The source should allow the author to state the map and the intended range; the provider should prove elementary arithmetic about that map. An unproved boundary term remains a goal. The same principle applies to products, finite partitions, and double counting.

Infinite sums require a different contract. A formal expression with infinitely many summands is not licensed for arbitrary rearrangement by the finite-sum plan. The appropriate convergence or summability hypotheses, and the exact theorem for the chosen notion of sum, must be present. This distinction belongs in the language's type-directed dispatch, not in a warning that appears only after an incorrect transformation.

7.5 Change of representation: a bridge with a return route

A calculation over $\mathbb{Q}$ or $\mathbb{R}$ may be easier than the corresponding goal over $\mathbb{N}$. A `calculate over Rat` plan should therefore specify an embedding, construct the transported statement, solve its arithmetic obligations, and prove that the transported conclusion implies the original one.

For addition and multiplication, the embedding is a homomorphism. For natural subtraction, transporting $a - b$ requires $b \leq a$ if it is to become the field difference. For natural division, transporting a/d to the field quotient requires exact divisibility, together with the relevant nonzero condition on d . A provider can infer these conditions from a known balance identity, but cannot assume them.

More general changes of viewpoint—sets to subtypes, coordinates to geometric objects, functions to their restrictions—should use the same pattern. A one-way implication can suffice for a goal reduction; an equivalence is required only when the surface claims equivalence. This avoids imposing unnecessarily strong obligations while keeping the direction precise.

8 Worked study II: Gaussian coefficients and finite summation

8.1 A recurrence-driven calculation

Write $\begin{bmatrix} n \\ k \end{bmatrix}_q$ for the division-free Gaussian coefficient in a semiring R , with $q \in R$. The relevant recurrence is

$$\begin{bmatrix} n+1 \\ j+1 \end{bmatrix}_q = \begin{bmatrix} n \\ j+1 \end{bmatrix}_q + q^{n-j} \begin{bmatrix} n \\ j \end{bmatrix}_q, \quad (8.1)$$

with $\begin{bmatrix} n \\ 0 \end{bmatrix}_q = 1$ and $\begin{bmatrix} n \\ n+1 \end{bmatrix}_q = 0$. Here $n - j$ is natural-number subtraction. The first summation theorem in the inspected source uses exactly the noncommutative-compatible factor order.[4]

For a weight function $w : \mathbb{N} \rightarrow R$, the desired identity is

$$\sum_{k=0}^{n+1} \begin{bmatrix} n+1 \\ k \end{bmatrix}_q w(k) = \sum_{k=0}^n \begin{bmatrix} n \\ k \end{bmatrix}_q w(k) + \sum_{j=0}^n q^{n-j} \begin{bmatrix} n \\ j \end{bmatrix}_q w(j+1). \quad (8.2)$$

All sums in this section are finite, and empty ranges have the usual empty-sum interpretation.

An explicit derivation shows precisely what a compact plan must reconstruct. Splitting the zero term and writing $k = j + 1$ gives

$$\sum_{k=0}^{n+1} \begin{bmatrix} n+1 \\ k \end{bmatrix}_q w(k) = w(0) + \sum_{j=0}^n \begin{bmatrix} n+1 \\ j+1 \end{bmatrix}_q w(j+1).$$

Substituting equation (8.1) and distributing on the right gives

$$w(0) + \sum_{j=0}^n \left[\begin{matrix} n \\ j+1 \end{matrix} \right]_q w(j+1) + \sum_{j=0}^n q^{n-j} \left[\begin{matrix} n \\ j \end{matrix} \right]_q w(j+1).$$

The first two pieces combine into $\sum_{k=0}^n \left[\begin{matrix} n \\ k \end{matrix} \right]_q w(k)$: the missing lower boundary is supplied by $\left[\begin{matrix} n \\ 0 \end{matrix} \right]_q = 1$, and the extra upper boundary vanishes by $\left[\begin{matrix} n \\ n+1 \end{matrix} \right]_q = 0$. This proves equation (8.2). No multiplication factors have been commuted.

A possible outline is therefore:

```
proof
  split the left sum at k=0
  reindex its remaining range via k=j+1
  expand q_pascal at (n+1,j+1)
  distribute the finite sums
  align the unshifted range using G(n,0)=1, G(n,n+1)=0
  conclude by additive_normalization
end
```

The phrases selecting subexpressions would be elaborated to stable expression locations, not fuzzy searches through printed strings. An implementation should allow explicit labels whenever the location is not unique.

8.2 Why the ambient algebra matters

In a semiring, addition is commutative but multiplication need not be. Rearranging the three displayed additive pieces is legitimate; changing $\left[\begin{matrix} n \\ k \end{matrix} \right]_q w(k)$ into $w(k) \left[\begin{matrix} n \\ k \end{matrix} \right]_q$ requires a separate commutation proof. Likewise, a normalization profile for a commutative ring cannot be silently used in a noncommutative semiring.

The second summation form in the source has a different placement of the power of q and a stronger ambient type-class assumption.[4] A language should preserve the assumptions of the chosen theorem rather than “simplify” both versions to the same prose. Selecting a recurrence is part of the proof, because recurrences with similar notation may have different algebraic requirements.

The exponent $n - j$ is only manipulated over the bound $j \leq n$ in the displayed sum. Replacing it by an integer difference without a bridge, or extending the range while treating negative exponents as if they were natural exponents, would change the statement.

8.3 Commutation by a generalized induction

Suppose $qx = xq$. Then every $\left[\begin{matrix} n \\ k \end{matrix} \right]_q$ commutes with x . The mathematical proof can be presented without naming each low-level commutation constructor.

For $n = 0$, the coefficient is either 1 or 0, both of which commute with x . Assume all coefficients in row n commute with x . The zero-column coefficient in row $n + 1$ is 1. Every other coefficient is, by equation (8.1), the sum of a coefficient from row n and a product of a power of q with another coefficient from that row. Powers of q commute with x , and the collection of elements commuting with x is closed under addition and multiplication. Hence the result follows.

For completeness, the product closure requires no commutation between the two factors themselves. If $ax = xa$ and $bx = xb$, then

$$(ab)x = a(bx) = a(xb) = (ax)b = (xa)b = x(ab).$$

Addition is handled by distributivity. Powers follow by induction on the exponent. This is the small closure theory that the plan needs, not an unrestricted claim that all products commute.

The proposed source makes the crucial generalization explicit:

```

proof
  induct n generalizing k
  case base:
    conclude by zero_row using commute_zero, commute_one
  case step:
    split k into zero or successor(j)
    case zero:
      conclude by zero_column
    case successor(j):
      expand q_pascal
      conclude by commutation_closure
        using induction_hypothesis, commute(q,x)
end
    
```

The author’s explanatory burden is the recurrence and the closure argument. The elaborator’s burden is specializing the induction hypothesis at both j and $j + 1$, constructing the power proof, and combining the evidence in the right order.

9 Worked study III: exact arithmetic through double counting

9.1 The formula and the source’s organization

For $n \in \mathbb{N}$, let $s = \lfloor \sqrt{n} \rfloor$, equivalently Lean’s `Nat.sqrt n`. The public formula in `FloorSqrtSum.lean` is a natural-number equality

$$\sum_{k=1}^n \lfloor \sqrt{k} \rfloor = s(n+1) - \frac{s(s+1)(2s+1)}{6}. \quad (9.1)$$

On the right, both subtraction and division have their natural-number meanings. The inspected proof establishes a rational identity and then supplies the conditions that make the transport exact.^[6]

A different mathematical organization is useful for the language discussion. It makes the divisibility and subtraction conditions consequences of a counting balance, rather than separate facts to guess. The following proof is an alternative presentation, not a claim of a new identity or a compiled replacement for the source.

9.2 A complete counting proof

Put

$$T = \sum_{k=1}^n \lfloor \sqrt{k} \rfloor, \quad Q_s = \sum_{j=1}^s j^2.$$

Consider the finite set

$$A = \{(j, k) \in \mathbb{N}^2 : 1 \leq j \leq s, 1 \leq k \leq n, j^2 \leq k\}.$$

For a fixed $k \in [1, n]$, the admissible j are exactly $1, \dots, \lfloor \sqrt{k} \rfloor$. Indeed, $j^2 \leq k$ is equivalent to $j \leq \lfloor \sqrt{k} \rfloor$, and this upper bound is at most s . Thus $|A| = T$.

For a fixed $j \in [1, s]$, we have $j^2 \leq s^2 \leq n$. The admissible k are exactly $j^2, \dots, n$, so their number is $n + 1 - j^2$. Therefore

$$T = \sum_{j=1}^s (n + 1 - j^2).$$

Since $j^2 \leq n + 1$ for every term, adding the corresponding squares cancels each natural subtraction and gives the exact natural-number balance

$$T + Q_s = s(n + 1). \quad (9.2)$$

There is no appeal here to a field interpretation of truncated subtraction.

The familiar sum-of-squares identity can itself be expressed without division:

$$6Q_s = s(s + 1)(2s + 1). \quad (9.3)$$

To prove it, the case $s = 0$ is immediate. If it holds at s , then

$$6Q_{s+1} = s(s + 1)(2s + 1) + 6(s + 1)^2 = (s + 1)(s + 2)(2s + 3),$$

where the final equality is a polynomial identity. This is the induction step.

Let $P = s(s + 1)(2s + 1)$. Equation (9.3) implies $P = 6Q_s$, hence $P/6 = Q_s$ in $\mathbb{N}$. Equation (9.2) then implies $T = s(n + 1) - Q_s$. Combining them proves equation (9.1). When $n = 0$, both index sets are empty, and every displayed identity remains valid.

9.3 An outline that records the combinatorial idea

```
proof
  let s := isqrt(n)
  let T := sum(k in 1..n, isqrt(k))
  let Q := sum(j in 1..s, j^2)
  count pairs (j,k) with
    1 <= j <= s, 1 <= k <= n, j^2 <= k
    first by k: T
    then by j: sum(j in 1..s, n+1-j^2)
  have balance : T+Q = s*(n+1) by finite_sum_arithmetic
  have squares : 6*Q = s*(s+1)*(2*s+1) by sum_squares
  conclude by exact_natural_arithmetic using balance, squares
end
```

The `count` plan must generate finite fibers and prove their cardinalities. Its square-root adapter must supply the exact characterization of the integer square root. The final arithmetic provider uses the balance and a product divisible by 6; it does not apply a field solver to natural division.

This example also exposes an evaluation pitfall. The source is concise only after a double-counting library and the appropriate square-root facts exist. Their implementation and maintenance costs belong in the experiment. A skilled Lean author could use the same library and obtain a short proof without adopting `OUTLINE`.

9.4 Generalizing the idea into a reusable plan

For a finite relation $A \subseteq J \times K$, the core double-counting theorem is

$$\sum_{j \in J} \#\{k \in K : (j, k) \in A\} = \#A = \sum_{k \in K} \#\{j \in J : (j, k) \in A\}.$$

A provider can instantiate this theorem, then expose the two fiber-cardinality goals. It should not infer a fiber formula from numerical samples. Numeric experimentation can suggest that formula, but the resulting identity must be proved for the symbolic parameters.

The same plan supports binomial coefficient identities, divisor counts, and finite lattice-point arguments. The useful abstraction is the checked interchange of counting directions, not a special-purpose macro for one square-root formula.

10 A small core calculus and a relative soundness argument

10.1 Judgments and environments

Let Σ be a fixed Lean environment and Γ a well-formed local context. Write

$$\Sigma; \Gamma \vdash t : C$$

for the underlying typing judgment. A complete proof block has an elaboration judgment

$$\Sigma; \Gamma \vdash p \Downarrow (t : C).$$

The source theorem is elaborated first to a particular target C . The proof elaborator is then given that target; it does not return an alternative theorem statement.

For an incomplete block, maintain obligations of the form

$$o_i = (\Gamma_i \vdash ?m_i : C_i),$$

where each Γ_i contains the local identities and dependencies needed at that point. A metavariable is an implementation placeholder for a future term, not a new axiom in Σ . A completed declaration may contain none of these placeholders.

Some local contexts depend on earlier synthesized witnesses. The graph therefore records dependent scopes and substitutions, not merely a set of strings naming propositions. Edges must respect lexical scope, and the reconstruction graph must be acyclic. A theorem cannot use its own unfinished declaration as a premise unless a separately justified recursive or inductive construction permits that use.

10.2 Core constructions

The following core is sufficient to explain the logical contract. It is intentionally smaller than the surface language.

Exact term. If $\Sigma; \Gamma \vdash e : C$, then **exact** e elaborates to e at target C .

Introduction. For a target $\forall x : A, B(x)$, a block under $\Gamma, x : A$ proving $B(x)$ elaborates to $\lambda x : A. t$. Implication introduction is the propositional special case. Bound variables are fresh and cannot capture source names accidentally.

Intermediate assertion. If a first block gives $u : A$ and a continuation under $\Gamma, h : A$ gives $v : C$, with the final goal C fixed in the outer context, then

$$\text{have } h:A \text{ by } p; q \rightsquigarrow (\lambda h : A. v) u.$$

A local definition uses the analogous **let** construction and the corresponding substitution rule. Dependent intermediate data require their types to be tracked through substitution.

Existential introduction. For target $\exists x : A, P(x)$, a witness $w : A$ and proof $u : P(w)$ yield $\langle w, u \rangle$. A witness suggestion alone does not close the goal.

Existential elimination. Given $e : \exists x : A, P(x)$ and a continuation under $\Gamma, x : A, h : P(x)$ proving the fixed proposition C , obtain a proof of C by existential elimination. The temporary witness and its property do not escape their permitted scope. In Lean, elimination from a propositional existence assertion into arbitrary computational data is restricted; a separate choice-based construction has a different logical and computational status.[27]

Case analysis. A proof of $A \vee B$ and branch proofs of C under $h_A : A$ and $h_B : B$ yield C by disjunction elimination. A fact established in only one branch is not available outside it without a common, discharged conclusion.

Certified plan application. A plan supplies a reconstruction term r whose arguments are the typed child proofs, including dependent continuations where necessary. Once those child proofs are available, applying r must type-check at the original target. Induction, WLOG, and reindexing elaborate through this interface rather than becoming new kernel rules.

Definitional conversion and equality transport. Definitional equality may be used where the kernel permits it. A non-definitional change requires an equality, equivalence, or implication proof appropriate to the direction of the step. Pretty-printed similarity is not a conversion rule.

10.3 Relative soundness

Theorem 10.1 (Soundness of completed core elaborations). *Assume that Σ and Γ are well formed, every accepted atomic evidence term is checked at its exact claimed type, and every plan application uses a reconstruction term at its declared dependent function type. Then every completed derivation*

$$\Sigma; \Gamma \vdash p \Downarrow (t : C)$$

using the core constructions above yields $\Sigma; \Gamma \vdash t : C$.

Proof. Proceed by induction on the elaboration derivation. The exact-term case is its premise. Introduction follows from the typing rule for a dependent lambda. Intermediate assertions and definitions follow from function application and substitution. Existential introduction and elimination follow from their constructors and recursors, with the stated scope condition. Disjunction elimination follows from its recursor and the two induction hypotheses. In a plan application, the induction hypotheses supply the child terms at exactly the types required by the reconstruction expression; repeated dependent application yields the original target. Conversion is justified either by the underlying definitional-equality rule or by the explicit transport term. These exhaust the core cases. $\square$

This is a soundness argument for the specified construction discipline, not a machine-checked proof of an implementation that does not yet exist. Final kernel checking is retained as an independent acceptance barrier, so a bug in the implementation of a rule should be detected rather than excused by the paper argument.

The theorem is *relative* to Σ and its axioms. It does not establish consistency of arbitrary imported axioms, correctness of all referenced definitions as models of informal concepts, security of the execution environment, or fidelity to an unexpressed human intention. Those are separate requirements.

10.4 Three guarantees that should not be conflated

Logical validity means the fixed formal statement has an accepted proof relative to the displayed assumptions. *Plan conformance* means the implementation used the checked transformation specified by the source plan and solved its designated child obligations. *explanatory adequacy* means a human reader finds that plan illuminating.

The kernel directly addresses the first. A plan checker and a validated source-to-obligation trace can address the second. Neither automatically gives the third. Even counting occurrences of a lemma in a proof term does not show that the lemma is essential; a proof can contain an unused intermediate assertion. Lean’s proof irrelevance further cautions against treating proof-term identity as an account of cognition.

The interface should therefore distinguish a kernel-checked theorem from an outline whose required plans have also passed conformance checks. Claims of human readability or faithful representation of mathematical reasoning require an evaluation with readers, not an additional green icon.

11 A synthesis architecture inspired by Leant

11.1 What should be reused

Leant’s overview presents structural term synthesis and interactive proof assistance with Lean-side candidate verification. Its supported fragments and provider mechanisms make it a useful source of candidate terms, rather than a universal theorem prover for arbitrary mathematics. That is the right starting point for OUTLINE’s local reconstruction stage.[8]

Several inspected components suggest concrete architectural lessons.

The verification module groups alternative textual realizations of candidates, tries variants, and lets accepted groups consume the success quota. Its opaque `Verified` wrapper explicitly represents acceptance by the supplied callback; it is not, by itself, a kernel proof, a solver certificate, or behavioral evidence. This is a valuable separation of authority.[10]

The selected suggestion path in `src/Main.hs` proposes tactics using features of the printed goal, then tests them on the proof state; merely presenting a successful suggestion does not insert it into the proof history. The heuristic recognition can be imperfect without making its suggestions logically authoritative.[9]

The fragment representation can retain opaque material that search may transport without understanding its internal mathematics. This distinction matters: a synthesizer able to rearrange implications involving an analytic proposition does not thereby know the mean value theorem.[11]

The candidate-quality documentation distinguishes compactness from structural variety, elimination cost, and provider cost. A useful preference order need not be a sort on printed string length.[12] The history-replay helper separately distinguishes reuse, suffix replay, and full replay after a history change. This is session synchronization, not yet the frozen theorem artifact proposed below.[13]

The synthesis-internals documentation also stresses binding evidence to the actual accepted candidate and its semantic origin, and qualifying modeled behavioral results by the model and provider laws. OUTLINE should retain those distinctions instead of compressing every successful check into a single word such as “verified.”[14]

11.2 A typed interface between the frontend and search

The frontend should send an obligation package containing the elaborated target, an ordered local-context telescope, universe parameters, local instances, selected facts, permitted plans, search budget, and policy identifier. Local references should use Lean expression identities, not authority inferred from pretty-printed names.

A human-readable goal string remains useful for an external language model or for diagnostics. It is not the identity of the goal. If a candidate returns as source text, it must be elaborated again in the exact original context and checked against the exact target. If a candidate returns as an

expression or a certificate, it must pass the corresponding typed validation and reconstruction.

An incremental adapter could continue using Leant’s existing Haskell engines and supported fragment translation. A later, Lean-native adapter could reduce duplicated representations and expose richer dependent contexts. Neither path requires discarding the working synthesis engines or pretending that the full Lean type theory has suddenly become a complete search fragment.

11.3 A portfolio of narrowly contracted providers

The first providers should be small and predictable. Structural synthesis assembles implications, products, disjunctions, witnesses available from providers, and local hypotheses. Simplification handles an explicitly scoped set of equations. Arithmetic providers handle identified algebraic theories. A theorem-application provider retrieves and instantiates named library facts. Domain plans, such as lower-slope reasoning or finite reindexing, create structured subgoals.

A learned model can propose a lemma, witness, induction motive, index map, or plan. An external solver can propose a certificate. Neither should have permission to alter the theorem header, install an axiom, or mark a goal solved. The trusted acceptance path receives only a candidate and asks for evidence of its type.

Search is bounded at several levels: number of generated candidates, total runtime, memory, recursion depth, and per-provider fuel. A success quota alone is not a sufficient resource bound, since an engine might reject an unbounded sequence of candidates before producing any accepted one.

11.4 A scheduler with explicit outcomes

```
reconstruct(obligation, policy, budget):
  freeze the target and context of obligation
  try a matching cached proof; recheck it in this environment
  select providers permitted by the source plan and policy
  for proposal in a bounded, interleaved provider stream:
    if proposal is a complete proof candidate:
      elaborate against the frozen target
      check the resulting term and its evidence policy
      retain it only if all checks succeed
    if proposal is a plan with child obligations:
      validate the reconstruction interface
      solve children in their own dependent scopes
      check the assembled term at the frozen target
  return the best admissible proof, or an explicit failure status
```

Listing 2: Algorithmic sketch, not an implementation.

Interleaving avoids letting one difficult search path consume the whole session. Completed candidates may be ranked by source fit, dependencies, replay cost, or structural diversity. These rankings affect what the author sees, not what counts as a proof.

The outcome space should include at least: *proved*, *open child obligations*, *budget exhausted*, *unsupported fragment*, *candidate rejected*, *policy rejected*, and *statement ambiguous*. A counterexample has an additional evidence status. None of the failure statuses means that the theorem is false.

11.5 Negative results and behavioral checks

A complete decision procedure for a specified fragment may establish noninhabitation or a countermodel *within that fragment*. Transferring such a result to the original Lean goal requires a faithful translation and the appropriate correctness direction. A bounded search that simply finds nothing has no such conclusion.

Behavioral testing has a similarly limited role. Suppose a synthesized function has type `List Nat -> List Nat`. Successful elaboration establishes that type, not a property such as preserving length. Testing selected inputs provides observations, not a proof of the universal law. A symbolic length model can offer stronger modeled evidence, but a theorem about the actual Lean function still requires a valid bridge from that model and proofs of the provider assumptions.

The proof-language interface should make these distinctions visible. An empirical counterexample can be useful in exploration; a claimed formal refutation requires a checked proof of the negation or a checked counterexample under the actual hypotheses. The system should never turn “no candidate in the current window” into “no such term exists.”

11.6 Search for the missing mathematical bridge

When reconstruction fails, the next useful action is often not deeper blind search but a better lemma boundary. In the residual example, failure to connect a derivative bound to an endpoint inequality indicates the missing lower-slope theorem or one of its hypotheses. In the Gaussian example, inability to use the induction hypothesis at $j + 1$ indicates an insufficiently generalized motive.

The language can support a *bridge suggestion*: propose an intermediate assertion, show how it would reduce the original goal, and separately attempt to prove it. Accepting the assertion does not assume it. The obligation graph retains both the bridge and the argument using it. This is a disciplined form of lemma discovery rather than an invisible expansion of the theorem’s assumptions.

12 Trust, semantic fidelity, and reproducible artifacts

12.1 The acceptance boundary must be narrower than the build process

Lean’s official validation guidance distinguishes ordinary kernel acceptance, transitive axiom inspection, rechecking stored proofs, and protection against actively malicious metaprograms. It also emphasizes that a checked formal statement must still match the intended informal statement.[25] OUTLINE should adopt that layered view rather than claiming that “uses Lean” alone resolves every trust question.

The proposed acceptance condition is:

$$\begin{aligned} \text{Accept}(p, C, \Sigma, \pi) \iff & \text{all scoped holes are filled and the proof term checks at } C, \\ & \text{the checked statement is the locked statement,} \\ & \text{all transitive assumptions satisfy policy } \pi, \\ & \text{the required plan-conformance checks pass.} \end{aligned} \tag{12.1}$$

The checks in this formula are a specification. A production implementation needs an independently protected path for performing them, especially when third-party code or model-generated metaprograms are involved.

12.2 An explicit axiom and computation policy

A practical standard-mathematics profile can allow the usual Lean principles `propext`, `Classi cal.choice`, and `Quot.sound`, while rejecting `sorryAx` and unapproved axioms in the theorem’s transitive dependencies. The profile is not “axiom-free.” A choice-restricted profile would need a different allowlist and library selection, not merely a switch hiding the word `classical`.^[26, 27]

Native evaluation needs separate treatment. The current reference explains that `native_decide` introduces a bespoke axiom for an invocation, while older compiler-trust mechanisms also exist. Thus a policy that merely rejects one historical axiom name is inadequate.^[26] The standard profile should reject any nonallowlisted computational axiom; an explicit elevated-trust profile can report exactly what native computations are being trusted.

External computation can remain outside the logical trusted base when it produces a certificate checked by a proved checker and the resulting theorem is reconstructed within the accepted proof policy. This is different from accepting a solver’s Boolean answer or trusting arbitrary compiled evaluation. A small certificate can be useful, but its size alone is not a correctness argument.

12.3 A semantic policy for total operations

Lean’s mathematical operations are not all partial in the informal sense. Field division is a total operation, and natural subtraction is truncated.^[24, 28] The frontend must therefore distinguish three things: whether an expression is well typed, whether a proposed algebraic law applies to it, and whether a domain convention in the mathematical exposition is satisfied.

Writing a/b does not by itself prove $b \neq 0$. Canceling b generally requires that proof. Likewise, the existence of a term $f'(x)$ in the formal language does not supply differentiability at x for an analytic theorem. A context template may explicitly request such hypotheses; it may not extract them from the visual appearance of the notation.

A project may adopt a stricter authoring convention under which certain notations generate domain obligations. Those obligations must still be discharged or appear in the theorem’s declared hypotheses. They cannot disappear merely because the underlying Lean operation happens to be total.

12.4 Freeze successful proofs, not just successful prompts

The interactive source is an exploration artifact. A frozen release should additionally contain the exact statement, accepted proof terms or proof-producing declarations, dependency manifest, policy, source-to-obligation map, and evidence audit. It should not require an external model service, a nondeterministic theorem search, or a remote solver merely to validate an unchanged accepted theorem.

A versioned artifact bundle could contain:

```
proof.outline           -- author-facing source
proof.expanded.lean     -- concrete generated declarations
proof.lock.json         -- toolchain, dependencies, policies
proof.map.json          -- source spans and obligation identities
proof.audit.json        -- assumptions and acceptance results
```

These are proposed formats. The expanded Lean file may be less pleasant to read than the outline; its purpose is reproducibility and debugging. Huge subterms should be shared through generated auxiliary declarations rather than repeatedly pasted into a single expression.

The lock must include more than a textual theorem hash. It should identify the Lean toolchain,

imported declaration environment, dependency revisions, selected notation and instances, provider versions, and the relevant checker configuration. A hash is useful for detecting changes and selecting a cache entry; it is not evidence that the entry proves the new goal. Cached terms are rechecked in the actual environment.

12.5 Two maintenance modes

In *replay mode*, unchanged artifacts use saved evidence and prohibit exploratory search. Failures indicate a mismatch or a checking problem. In *repair mode*, changed dependencies or statements may trigger search again, but the result is a new artifact with a semantic diff and a fresh audit.

A source-level change in notation that leaves the elaborated statement definitionally identical need not invalidate the mathematical proof. A changed definition behind the same printed name may invalidate it even when the source text looks unchanged. Similarly, a different type-class instance can change interpretation. The diff should therefore report changes to elaborated statements and selected dependencies, not just lines of text.

A tactic trace is useful for reproducing exploration, but is not always the best archival evidence: later changes in simplification sets or search heuristics can alter its behavior. Retaining a checked proof term or sufficiently explicit reconstruction removes much of that avoidable instability, though kernel checking and dependency availability remain real costs.

12.6 Operational security is not logical soundness

A tactic or elaborator may execute arbitrary host-level code. Therefore model-generated plugins and untrusted imports should be explored in a sandbox with restricted filesystem and network access. The final validation path should not execute that code as an unavoidable part of checking the proof.

For adversarial scenarios, the official Lean guidance describes separating a trusted challenge statement from the submitted proof, sandboxing the build, validating exported representations, and rechecking with protected checkers.^[25] OUTLINE’s statement lock fits that architecture, but a lock file alone does not implement it. Serialization validation, declaration loading, the checker, the sandbox, and the machine remain part of the operational trust story.

13 The authoring experience: progressive disclosure without concealment

13.1 Three synchronized views

The default view should show the mathematical outline and a small status indicator for each checkpoint. A second view should show its generated obligations, selected lemma applications, and local assumptions. A third should show the precise Lean goal and evidence. Moving between views should preserve source location and scope.

This is not an instruction to hide every difficult goal. An unresolved side condition is important information and should be promoted into the default view. Once it is discharged by routine evidence, it can collapse again. The author should never have to guess whether a missing detail was proved, assumed, skipped, or merely not yet checked.

For the residual proof, expanding the slope checkpoint might show the left and right applications, endpoint memberships, and the available derivative hypothesis. Expanding the field step might show that $m \neq 0$ was obtained from $m > 0$. The outline remains short precisely because these are available on demand rather than repeated in the main narrative.

13.2 Useful error messages are mathematical

An illustrative diagnostic for a failed reindexing might be:

```
Reindexing did not discharge its upper boundary.
Map: j |-> j+1
Remaining term: G(n,n+1)*w(n+1)
Needed: this term is zero, or an explicit correction term.
The original sum has not been changed.
```

For induction, the corresponding message might be:

```
The current induction hypothesis is available at k only.
The recurrence also requires the hypothesis at k+1.
Suggested source change: induct n generalizing k
This is a stronger motive; review before accepting.
```

These are proposed diagnostics, not transcripts from a running implementation. They state the failed contract and the proposed mathematical repair. They avoid the misleading impression that a failed tactic is evidence against the theorem.

13.3 Suggestions should be diverse at the level of ideas

A menu containing five syntactically different versions of the same lemma application provides little choice. More useful alternatives might be: apply a mean-value theorem; prove monotonicity first; establish a residual bracket directly; or reduce to an existing inverse-error theorem when an inverse is already known.

The last alternative is not interchangeable with the first in the existence theorem: it introduces an assumption the theorem is intended to avoid. Candidate ranking should penalize or reject such mismatches with the declared plan and available hypotheses. “Interesting” suggestions should include their prerequisite obligations, not merely a persuasive sentence.

The author can then accept a mathematical route and let local synthesis fill in the details. This is closer to collaboration on an argument than to an endless sequence of tactic completions.

13.4 Keep the proof’s story honest

An outline generator might produce attractive prose that is not supported by the stored proof. A simple guard is to generate the prose from the accepted plan graph rather than asking a model to invent an explanation from the final theorem alone. Every explanatory sentence then has a location in that graph.

Even this does not establish pedagogical quality. Some intermediate assertions are redundant, some proof plans are needlessly indirect, and a faithfully described search-derived proof may still be hard to understand. The interface should permit refactoring the outline and regenerating its evidence, while retaining the distinction between a changed presentation and a changed statement.

14 What should be optimized, and how to test it

14.1 A multidimensional cost model

A proposed search objective can combine several costs:

$$J = \alpha C_{\text{author}} + \beta C_{\text{reader}} + \gamma C_{\text{search}} + \delta C_{\text{check}} + \varepsilon C_{\text{maintenance}}. \quad (14.1)$$

Here author cost measures annotations and corrective interactions; reader cost measures comprehension effort; search cost measures runtime and memory; checking cost measures proof replay; and maintenance cost measures sensitivity to controlled changes. The weights depend on the project. None of these quantities is accurately represented by source characters alone.

Some requirements should be hard constraints rather than weighted penalties: the statement must not drift, unsupported assumptions must not be added, the policy must be satisfied, and required plans must be honored. A slightly shorter proof is not a compensation for violating one of these constraints.

A practical candidate ranker can use proxies such as number of opaque theorem applications, dependency depth, proof sharing, and size of the unresolved obligation frontier. Those proxies are hypotheses about usefulness. Their relationship to actual comprehension must be measured rather than assumed.

14.2 A fair comparison has more than one Lean baseline

An evaluation should compare OUTLINE not only with the existing source but also with professionally refactored Lean using the same helper lemmas and automation. Otherwise the experiment confounds a language advantage with a library or proof-refactoring advantage.

A useful ablation sequence is: original source; refactored Lean; refactored Lean plus new reusable lemmas; the controlled syntax alone; syntax plus local synthesis; syntax plus certified domain plans; and the full system with freezing and diagnostics. The same theorem statements and axiom policies must be used throughout.

The worked examples in this article can seed a benchmark, but cannot by themselves establish a result. They were selected to expose design opportunities, so reporting success on them alone would favor the design by construction. Held-out theorem families and unrelated modules are necessary.

Recent benchmark-audit work reinforces the importance of statement fidelity. Ammanamanchi, Bhat, and Biderman report specification defects and evaluation failures in Lean theorem-proving benchmarks, including missing hypotheses and incorrect translations. Their findings motivate independent review of the benchmark statements and evaluation harness, not just checking submitted proofs.[29]

14.3 Measurements to report

Raw line counts should not be compared after reformatting one language densely and the other generously. Token counts are somewhat more reproducible, but still do not measure mathematical insight. Count the accepted theorem statement, user-written proof body, helper declarations, and imported specialized infrastructure explicitly.

A family-level train/test split is important when learning or retrieval is involved. A held-out theorem whose proof is a renamed copy of a training lemma is not a meaningful test of new reasoning. For repository-scale experiments, near-duplicate formulas, shared generated templates, and helper-theorem leakage should be recorded.

Dimension	Measurement	Important control
Author effort	Time, edits, explicit annotations, failed suggestions	Match experience and theorem difficulty
Readability	Questions answered, errors detected, explanation time	Blind or counterbalanced presentation
Conciseness	Tokens and semantic checkpoints in theorem and helpers	Count shared library additions separately
Reconstruction	Success rate, latency distribution, timeout rate	Fixed budgets, seeds where applicable, toolchain
Replay	Checking time, memory, artifact size, offline success	Disable search and external services
Maintenance	Repairs after controlled source and library changes	Distinguish target changes from harmless refactoring
Safety	Rejection of invalid transformations and policy violations	Include adversarial and missing-premise tests

Table 4: A proposed measurement plan. No numerical outcomes are claimed in this article.

14.4 Test the failure boundary as carefully as success

A minimum negative suite should include cancellation without nonzeroness, natural subtraction interpreted as integer subtraction, symmetry with asymmetric hypotheses, reindexing that loses a boundary, an unjustified permutation of infinite summands, an insufficient induction motive, a candidate using `sorryAx`, and a stale cache entry for a changed definition.

The expected response is not always “false theorem.” Some inputs are true but their proposed plan is invalid; some are unsupported by the current provider; some simply exceed the budget. The test oracle should distinguish these outcomes. A system that labels every rejected proof false is unsafe as a mathematical assistant even when it never admits a bad proof.

14.5 Evaluate the cognitive claim directly

The claim that the language better represents mathematical reasoning is an empirical claim about authors and readers. A study should ask participants to reconstruct the proof’s main idea, identify which hypotheses are used where, predict the next step, and detect deliberately removed assumptions. Include both experienced Lean users and mathematicians without substantial Lean experience.

A counterbalanced design can present equivalent arguments in ordinary mathematical prose, idiomatic Lean, and OUTLINE. The study should separate learning the new syntax from understanding the argument. It should also measure whether easy access to hidden obligations improves error detection or merely encourages readers to ignore them.

No outcome is guaranteed. A controlled language may reduce some burdens while adding others; a domain plan may be concise for its designer but obscure to a new user. The evaluation should be prepared to find that a good library lemma is preferable to a new syntax form for some proof families.

15 An implementation path that can be attempted incrementally

15.1 Start as a Lean extension, not a replacement foundation

The lowest-risk first implementation is a Lean package providing syntax, elaborators, an obligation data model, and a small editor extension. Lean already offers extensible syntax, elaboration, tactics, and checked proof terms.[24] Retaining that infrastructure permits direct use of ProveIt’s

mathematics and ordinary Lean proof blocks.

A standalone parser or external Haskell frontend remains possible, particularly for reusing Leant, but should communicate through a narrow typed protocol. Maintaining an independent parser, instance resolver, and theory of dependent contexts would otherwise create substantial duplicated complexity. There is no reason to invent a new kernel to obtain the proposed surface improvements.

15.2 Milestone 1: a vertical slice with no magical automation

Implement explicit theorem headers, local definitions, named assertions, **using only**, and native Lean escape blocks. Each checkpoint should elaborate to a fixed target and a scoped hole. Initially, only supplied Lean terms and a small deterministic tactic profile solve those holes.

The exit criterion is not broad theorem coverage. It is that a theorem written with checkpoints produces the same statement as its explicit Lean counterpart, every obligation is accounted for, failed checking never publishes a theorem, and the expanded proof can be replayed without the authoring interface.

15.3 Milestone 2: connect synthesis and freeze accepted evidence

Adapt Leant's candidate production and verified suggestion workflow to the obligation protocol. Keep callback acceptance, Lean type checking, behavioral evidence, and policy acceptance as separate stages. Add multiple resource bounds and failure statuses before expanding the search portfolio.

Implement frozen evidence for completed checkpoints, statement and dependency comparison, and selective invalidation after edits. Test undo and replacement histories as well as append-only sessions. Leant's simple replay distinction is useful here, but theorem-artifact validity needs stronger environment and target checks than chronological prefix matching alone.

15.4 Milestone 3: three domain plans

The first domain plans should be deliberately limited: WLOG under a supplied symmetry; finite-range reindexing with a supplied map; and lower-derivative-bound reasoning on supplied intervals. These are motivated by the inspected examples and have clear contracts.

Implement one plan end to end, including diagnostics and negative tests, before adding more surface vocabulary. A plan should have a documented reconstruction theorem, a description of its context behavior, resource limits, and an example where it correctly refuses to proceed.

Generalized induction is another early candidate because Lean supplies the relevant recursors. Its interface should require or explicitly confirm the motive rather than infer hidden generalizations from failed searches.

15.5 Milestone 4: a domain library and evaluation

Add counting, recurrence, and arithmetic-transport adapters. Refactor a small ProveIt sample in ordinary Lean alongside the OUTLINE versions. Keep all source snapshots and artifact policies pinned, publish the benchmark generator and negative suite, and report failures as well as successful examples.

Only after these measurements should the project decide whether to build a richer mathematical phrase grammar. A large natural-language vocabulary without reliable scope, evidence, and diagnostics would be an attractive demonstration but a fragile proof language.

15.6 Open design questions

The principal unresolved engineering question is how far reconstruction can be made predictable without sacrificing usefulness. Context saturation, theorem retrieval, and higher-order unification can all make a short checkpoint expensive. A second question is how to preserve semantic source mappings across proof-term sharing and normalization. A third is how to let authors extend the plan library without making the default environment unpredictable.

There is also a conceptual limit. No fixed set of plans captures every useful mathematical argument, and not every insight has a concise formal decomposition. The language should therefore be extensible and should expose native Lean when needed. Its success would be a reduction in routine authorial burden while preserving meaningful structure, not the elimination of difficult formalization work.

16 Conclusion

A better mathematical proof language should not ask its authors to become less precise. It should ask them to be precise about the right things: the statement, the important intermediate facts, the choice of representation, and the transformations that carry the argument.

The ProveIt examples suggest concrete opportunities. A residual proof can be organized around a simultaneous endpoint bracket. A symmetric estimate can expose its symmetry once. A Gaussian-binomial calculation can name its finite index map and boundary behavior. A commutation proof can state its induction motive and closure argument. A floor-square-root identity can derive exact arithmetic conditions from a counting balance. These are mathematical units of explanation, not merely abbreviations for tactic strings.

Leant suggests how to connect those units to search without confusing a candidate with evidence. The proposed architecture carries that separation through the entire workflow: fixed formal statements, scoped obligations, bounded candidate generation, exact reconstruction, policy-aware checking, and replayable artifacts.

The recommended next step is therefore modest in implementation scope but strict in contract: build a Lean-embedded checkpoint language with explicit obligations and frozen proofs, then implement a few certified plans and evaluate them against equally well-refactored Lean. The aim is not a proof that is shorter at any cost. It is a proof whose visible structure explains why the result is true, while every omitted detail remains recoverable as checked mathematics.

A Source inventory and reproducibility notes

The review is reproducible at the two commits identified in section 1. The following inventory records the particular evidence used. It does not claim that every declaration in a listed repository was inspected or that any repository was compiled during this work.

Provelt

The first four file names below are relative to `Analysis/FabiusFunction/Lean/FabiusFunction/`.

File	Reviewed material and relevance
<code>Basic.lean</code>	Bounded values, the real-valued view, endpoint properties, <code>IsFabius</code> , and symmetry. Used to distinguish safe local projection from changing mathematical objects.[2]
<code>MeanValueBracket.lean</code>	Absolute-value derivative brackets, <code>exists_eq_in_residual_interval</code> , and right-inverse corollaries. Used for the residual example and checked symmetry reduction.[3]
<code>QPascalSummation.lean</code>	Both finite summation forms and <code>Commute.gaussianBinomial_left</code> . Used for range transformations, factor order, and generalized induction.[4]
<code>Arithmetic.lean</code>	Selected early binomial and factorial identities, including <code>two_mul_choose_two_add</code> and <code>factorial_two_mul_eq</code> . Used as secondary recurrence and finite-product examples.[5]
<code>FloorSqrtSum.lean</code>	The file under <code>NumberTheory/IntegerSums/Lean/IntegerSums/</code> . Rational intermediate formula, square-root step lemmas, divisibility, natural subtraction, and final cast transfer.[6]

Leant

File or document	Reviewed material and relevance
<code>README.md</code>	Overall synthesis/proof workflow and stated fragment boundaries.[8]
<code>src/Main.hs</code>	Selected advisory suggestion and tactic-probing implementation, not a review of the entire large module.[9]
<code>src/Leant/Synth/Verification.hs</code>	Candidate groups, variant acceptance, success quota, and the explicitly limited meaning of the wrapper.[10]
<code>src/Leant/Synth/Fragment.hs</code>	Selected fragment representation and refusal logic for opaque or unsupported content.[11]
<code>docs/candidate-quality.md</code>	Ranking before cutoffs; structural cost and diversity.[12]
<code>src/Leant/Synth/Replay.hs</code>	Pure choice among reuse, suffix replay, and full replay.[13]
<code>docs/synth-internals.md</code>	Selected architecture documentation on semantic origins, verification, and modeled evidence.[14]

The external literature and documentation were consulted as available on September 5, 2026. The moving Lean reference then identified itself as covering version 4.34.0-rc2. That documentation version is not asserted to be the toolchain of either inspected repository; an implementation must use its target repository’s pinned toolchain.[24]

The accompanying archive contains this self-contained L^AT_EX source, its rendered PDF, a README with build instructions and limitations, and a plain-text source manifest. It contains no proposed compiler binary, no test execution report for the language, and no measured performance data.

B A minimal grammar and obligation record

A deliberately small first grammar

The following grammar describes an implementable *initial subset*. Richer mathematical phrases elsewhere in the article are proposed macro schemas over this subset and the certified-plan interface; they are not all claimed to be covered by this grammar. Terms, binders, and native tactic blocks are delegated to Lean’s parser in a controlled environment.

```
document ::= "theorem" name binders ":" term
          "proof" block "end"
block     ::= statement* conclusion
statement ::= "let" name [ ":" term ] "!=" term
          | "have" name ":" term "by" justification
          | "obtain" pattern "from" term
justification ::= "lean" "{" lean_tactics "}"
          | "plan" name arguments [fact_scope]
          | "search" [fact_scope] [budget]
fact_scope  ::= "using" ["only"] name ("," name)*
budget      ::= "budget" natural
conclusion  ::= "exact" term
          | "conclude" "by" justification
```

Listing 3: Illustrative grammar for the first implementation.

Statement boundaries use the host language’s layout rules or explicit delimiters. A native tactic block is an exploration mechanism; freezing records its result rather than requiring the block to be rerun for every replay. Case analysis and induction can be added as structured commands with explicit branch blocks once scope tracking is established.

An obligation is more than its displayed proposition

A proposed internal record contains:

```
Obligation {
  id, sourceSpan, parentPlan,
  environmentIdentity, localContext, localInstances,
  universeParameters, targetExpr,
  predecessorObligations, scopeBoundary,
  permittedFacts, requiredPlan, policy, resourceBudget,
  state, acceptedEvidence
}
```

The context and target fields denote structured expressions. The identity fields support book-keeping and invalidation, not proof acceptance. `acceptedEvidence` is absent until the designated checks have succeeded. If a predecessor supplies a dependent witness, the record also tracks the corresponding substitution and the scope within which it is valid.

The state machine should forbid transitions directly from “candidate proposed” to “published theorem.” It should also distinguish the temporary success of a local tactic from completion of the enclosing theorem and its transitive audit.

C Evidence states and a negative-test specification

Evidence-state distinctions

State	What it establishes, and what it does not
Candidate	A term, tactic, map, or plan has been proposed. No proof claim follows.
Callback accepted	A designated test accepted a candidate. Its meaning depends on that callback and is not automatically a kernel guarantee.
Elaborated	The candidate has been interpreted against the target context. Remaining obligations, hidden dependencies, and final checking still matter.
Kernel checked	A concrete term has been checked at the fixed formal target in the designated environment. This is relative to that environment and its assumptions.
Policy accepted	The transitive assumptions and computation mechanisms satisfy the chosen allowlist and artifact policy. This does not prove the consistency of arbitrary mathematical hypotheses.
Plan conformant	The required source plans have the designated expansion and verified child obligations. This is not a guarantee that the proof is the best explanation.
Published	The required checks are complete and the evidence bundle is frozen. A later changed statement or environment requires revalidation.

The separate state corresponding to callback acceptance is directly motivated by the carefully limited interface in Leant’s verification module; the remaining lifecycle is proposed here.^[10]

Negative and boundary tests

ID	Input or attempted move	Required behavior
T1	Cancel a in $ax = ay$ with no proof that $a \neq 0$.	Leave a nonzeroness obligation or use another valid argument; do not silently conclude $x = y$.
T2	Rewrite $(n - 1) + 1 = n$ for all $n \in \mathbb{N}$.	Reject the unrestricted rewrite; $n = 0$ is a counterexample.
T3	Transport natural $3/2$ to the field quotient $3/2$.	Refuse the exact-division bridge because the divisibility condition fails.
T4	Swap x, y while retaining an asymmetric condition $x = 0$.	Demand a context transport or reject that WLOG plan; do not infer that the target itself is false.
T5	Shift $k = j + 1$ while losing an endpoint.	Produce the missing boundary obligation or correction term.
T6	Reindex a sum with a noninjective map and ignore multiplicity.	Refuse the bijection plan or require a fiber/multiplicity formulation.

ID	Input or attempted move	Required behavior
T7	Apply the finite-reindex plan to an infinite, conditionally convergent series.	Require the separate applicable convergence/rearrangement theorem.
T8	Use an induction hypothesis at $k + 1$ when only the instance at k exists.	Suggest and expose a stronger motive; never grant the extra hypothesis silently.
T9	Apply the residual-root plan without interval containment.	Preserve the domain obligation. The clamped-function example in section 6 supplies a genuine counterexample to the weakened theorem.
T10	Infer an upper absolute-value estimate from an upper derivative bound that allows negative derivatives.	Require a valid absolute derivative bound or the needed monotonicity/sign assumptions.
T11	Candidate depends transitively on <code>sorryAx</code> or an unapproved fresh axiom.	Reject publication under the standard profile, even if local elaboration succeeds.
T12	A cached term has the same printed target but a referenced definition has changed.	Recheck against the actual changed environment and invalidate stale evidence.
T13	A search provider exhausts its budget on an opaque proposition.	Report exhaustion or unsupported structure, not noninhabitation.
T14	An existential witness introduced in a branch escapes into an unrelated outer definition.	Enforce scope and Lean’s elimination restrictions; require an appropriate explicit construction.
T15	The residual radius is zero, a finite range is empty, or two WLOG variables are equal.	Handle the legitimate degenerate case without introducing unnecessary strict inequalities.
T16	A source phrase has two genuinely different formal interpretations.	Resolve from declared context or request a choice before proof search; do not select by provability.

This is a test specification, not a report that these tests were run. The future implementation should also include positive neighboring cases for each refusal, so that safety does not become an excuse for rejecting every useful transformation.

References

- [1] Vladimir Reshetnikov and contributors. *ProveIt*, reviewed repository revision [24ce8bd743eaab64a91ce90725ea00f498d319d2](#). [Commit record](#). Accessed September 5, 2026.
- [2] Vladimir Reshetnikov and contributors. *ProveIt: Basic.lean*. [Pinned source in the FabiusFunction module](#). See the bounded function, real-valued view, and `IsFabius` declarations.
- [3] Vladimir Reshetnikov and contributors. *ProveIt: MeanValueBracket.lean*. [Pinned source](#). See the absolute-value bracket theorems, `exists_eq_in_residual_interval`, and the right-inverse corollaries.
- [4] Vladimir Reshetnikov and contributors. *ProveIt: QPascalSummation.lean*. [Pinned source](#). See both summation theorems and the Gaussian-coefficient commutation lemmas.
- [5] Vladimir Reshetnikov and contributors. *ProveIt: Arithmetic.lean*. [Pinned source](#). Selected early binomial, triangular-number, and factorial identities.
- [6] Vladimir Reshetnikov and contributors. *ProveIt: FloorSqrtSum.lean*. [Pinned source](#). Natural-number summation formula and its rational proof with exact cast transfer.
- [7] Vladimir Reshetnikov and contributors. *Leant*, reviewed repository revision [c36adf114035fb2fba6c276b63944cc613b31668](#). [Commit record](#). Accessed September 5, 2026.
- [8] Vladimir Reshetnikov and contributors. *Leant: README*. [Pinned project overview](#). Synthesis, interactive proof workflow, and supported-fragment overview.
- [9] Vladimir Reshetnikov and contributors. *Leant: src/Main.hs*. [Pinned source](#). Selected goal-dependent tactic suggestion, probing, and proof-session code.
- [10] Vladimir Reshetnikov and contributors. *Leant: src/Leant/Synth/Verification.hs*. [Pinned source](#). Grouped candidate variants, callback acceptance, and its limited evidence meaning.
- [11] Vladimir Reshetnikov and contributors. *Leant: src/Leant/Synth/Fragment.hs*. [Pinned source](#). Fragment representation and selected opaque-content/refusal handling.
- [12] Vladimir Reshetnikov and contributors. *Leant: Candidate Quality Before Verification*. [Pinned documentation](#).
- [13] Vladimir Reshetnikov and contributors. *Leant: src/Leant/Synth/Replay.hs*. [Pinned source](#). The `planReplay` history-synchronization helper.
- [14] Vladimir Reshetnikov and contributors. *Leant: Synthesis Internals*. [Pinned documentation](#). Selected material on candidate origins, reconstruction, and modeled evidence.
- [15] Markus Wenzel and the Isabelle project. *Isar—Intelligible Semi-Automated Reasoning*. Official project overview and primary-publication collection. <https://isabelle.in.tum.de/Isar/>.
- [16] Adam Naumowicz and Artur Korniłowicz. A Brief Overview of Mizar. In *Theorem Proving in Higher Order Logics*, LNCS 5674, pp. 67–72, 2009. doi:10.1007/978-3-642-03359-9_5.
- [17] Makarius Wenzel. *Isabelle/Naproche for Automatic Proof-Checking of Ordinary Mathematical Texts*. June 11, 2019. <https://sketis.net/2019/isabelle-naproche-for-automatic-proof-checking-of-ordinary-mathematical-texts>.
- [18] Patrick Massot and contributors. *Verbose Lean 4: Natural Language Tactics to Teach Mathematics Using Lean 4*. Project repository, consulted September 5, 2026. <https://github.com/PatrickMassot/verbose-lean4>.
- [19] Patrick Massot. Teaching Mathematics Using Lean and Controlled Natural Language. In *ITP 2024*, LIPIcs 309, pp. 27:1–27:19, 2024. doi:10.4230/LIPIcs.ITP.2024.27.
- [20] Jannis Limperg and Asta Halkjær From. Aesop: White-Box Best-First Proof Search for Lean. In *CPP 2023*, pp. 253–266, 2023. doi:10.1145/3573105.3575671. See also the [official Aesop repository](#).

- [21] Jasmin Blanchette, with contributions from Martin Desharnais, Lawrence C. Paulson, and Lukas Bartl. *Hammering Away: A User’s Guide to Sledgehammer for Isabelle/HOL*. Isabelle2025 manual, March 13, 2025. <https://isabelle.in.tum.de/website-Isabelle2025/dist/Isabelle2025/doc/sledgehammer.pdf>.
- [22] Alan Bundy. Proof Planning. *AIPS 1996 Proceedings*, pp. 261–267, 1996. <https://cdn.aaai.org/AIPS/1996/AIPS96-033.pdf>.
- [23] Sage Binder, Hanna Lachnitt, and Katherine Kosaian. *Apply2Isar: Automatically Converting Isabelle/HOL Apply-Style Proofs to Structured Isar*. Preprint, arXiv:2603.07771, 2026. <https://arxiv.org/abs/2603.07771>.
- [24] The Lean development team. *The Lean Language Reference*. Online reference, consulted September 5, 2026; the consulted moving version identified itself as 4.34.0-rc2. <https://lean-lang.org/doc/reference/latest/>.
- [25] The Lean development team. *Validating a Lean Proof*. In the online Lean Language Reference, consulted September 5, 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [26] The Lean development team. *Axioms*. In the online Lean Language Reference, consulted September 5, 2026. <https://lean-lang.org/doc/reference/latest/Axioms/>. See also the `tactic` reference for computation and proof-producing tactics.
- [27] Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich. *Theorem Proving in Lean 4*, chapter “Axioms and Computation.” Online edition, consulted September 5, 2026. https://lean-lang.org/theorem_proving_in_lean4/Axioms-and-Computation/.
- [28] The Lean development team. *Natural Numbers*, section on arithmetic. In the online Lean Language Reference, consulted September 5, 2026. <https://lean-lang.org/doc/reference/latest/Basic-Types/Natural-Numbers/>.
- [29] Pawan Sasanka Ammanamanchi, Siddharth Bhat, and Stella Biderman. *Faults in Our Formal Benchmarking: Dataset Defects and Evaluation Failures in Lean Theorem Proving*. Preprint, arXiv:2606.29493, submitted June 28, 2026. <https://arxiv.org/abs/2606.29493>.